

ARYA MEHTA

• San Francisco Bay Area • (408) 829-4465 • aryamehta456@gmail.com • [LinkedIn](#) • [GitHub](#) • [Portfolio](#) • [Medium](#)

EDUCATION

San Jose State University — San Jose, CA

Jan 2025 – Dec 2026

Master of Science in Applied Data Science

OPEN-SOURCE CONTRIBUTION: *LangGraph(Python)*

Contributed an Agentic Commerce multi-agent architecture pattern to the official LangGraph repository.

TECHNICAL SKILLS

Languages: Python, Java, Go, C++, JavaScript/TypeScript, SQL

AI & Machine Learning: LangGraph, LangChain, RAG, Agentic Workflows, NLP, PyTorch, Scikit-learn, TensorFlow

Backend & Web: FastAPI, Spring Boot, Node.js, React, Next.js, REST APIs, gRPC, WebSocket, NGINX

Data Systems & Infrastructure: PostgreSQL, AWS Aurora, DynamoDB, MongoDB Atlas Vector Search, Redis, Kafka, Apache Spark, Airflow

Cloud & DevOps: AWS (EKS, S3, RDS, EC2), Docker, Kubernetes, Jira, Jenkins, Maven, GitHub Actions (CI/CD), Prometheus, Grafana

PROFESSIONAL EXPERIENCE

MeteoControl — Ahmedabad, India

Jan 2024 – Dec 2024

Software Engineer

- Shipped production features weekly across **Java/Spring Boot** services and **React/TypeScript** UIs for **10k+ users**, sustaining **99.9%+ availability** via Prometheus/Grafana alerts and on-call rotations.
- Refactored hot paths from **$O(n^2)$ → $O(n \log n)$** using B+ trees & hashing → **+40% throughput** and **~60% p95 latency** on Linux systems.
- Implemented resilience (circuit breakers, retries) and **blue-green/canary** deploys; automated **CI/CD** to cut release time **50%** with zero-downtime rollouts.
- Built Python/Bash tooling for deploy/monitor/log analysis, eliminating **~20 hrs/week** manual toil and reducing incidents **75%** via proactive troubleshooting.

Dupat Infotronicx Pvt. Ltd. — Ahmedabad, India

Aug 2022 – Aug 2023

Software Engineer Intern

- Developed an end-to-end **web + data platform** (React, FastAPI, Postgres/Redis) processing **1M+ records/day**; caching, pagination, and rate-limiting cut DB load **70%**.
- Re-architected ETL with memoization + batched I/O → **3x faster** processing and **~65% CPU**.
- Delivered REST APIs with ~95% test coverage; created dashboards/runbooks for on-call; collaborated with cross-functional teams in Agile sprints and presented technical insights to non-technical stakeholders.

PROJECTS

LayoverOS (MongoDB Agentic Hackathon)

Python, FastAPI, LangGraph, MongoDB Atlas, Next.js, Coinbase CDP

[GitHub Repository](#)

- Architected a multi-agent state machine using **LangGraph**, routing user intent via a Supervisor node to specialized autonomous agents (Scout, Flight, Bursar) to parallelize context execution and reduce response latency.
- Engineered a zero-hallucination retrieval pipeline on **MongoDB Atlas Vector Search**, combining 1024-dimensional **Voyage AI** embeddings with strict metadata pre-filters (terminal ID, open status) to guarantee 100% data accuracy.
- Implemented Agentic Commerce by integrating the **Coinbase Developer Platform (CDP)**, enabling the AI to autonomously construct transaction payloads and trigger **Generative UI** payment modals directly within the **Next.js** frontend.
- Hardened system reliability via **MongoDBSaver checkpointing** for persistent multi-turn memory, and designed a self-healing fallback mechanism to automatically reroute to raw database queries during LLM provider outages.

SJ Hopes - Collaborative Platform (2nd Prize among 300+ teams)

TypeScript, React, Next.js, Java, Spring Boot, MySQL

[GitHub Repository](#)

- Won 2nd place** implementing real-time collaboration using WebSocket, geospatial algorithms (Haversine formula), and efficient data structures (R-trees for location queries); 48-hour hackathon.
- Scaled application to support **1000+ concurrent users** using connection pooling, database sharding, and Redis caching; achieved **<100ms P95** response time under load.
- Achieved 95% test coverage with unit/integration tests; implemented Progressive Web App features including offline support and push notifications.
- Applied graph algorithms (**Dijkstra's**) for route optimization and collaborative filtering for recommendations; reduced memory footprint by **45%** using efficient data structures.

Distributed Data Processing Platform

Python, Airflow 2.x, Spark 3.5, FastAPI, Docker, Kubernetes (AWS EKS), Prometheus/Grafana, S3

[GitHub Repository](#)

- Orchestrated **5 TB+/day** ETL on **Kubernetes/EKS** using Airflow (**Kubernetes/Celery Executor**) and Spark-on-K8s; tuned partitioning & predicate pushdown → **~60% shuffle**, **~55% wall-clock**, infra cost **~40%** via autoscaling/right-sizing.
- Instrumented a **50+ rule data-quality framework** (nulls, uniqueness, ranges, regex, temporal) integrated into DAGs; exported metrics to **Prometheus** with **Grafana** SLO dashboards/alerts → **anomalies ~85%**, **job success 99.9%**.
- Shipped **FastAPI** microservice with **readiness/liveness** probes, **/metrics**, and **gunicorn+uvicorn**; maintained **p95 < 100 ms** on health/metrics paths; hardened reliability with **RBAC**, resource limits, retries, and idempotent task design.
- Automated **CI/CD** via **GitHub Actions** (tests + image builds for API/Spark), versioned **K8s manifests**, and runbooks for **Unix/Linux** on-call **troubleshooting** and rollback.